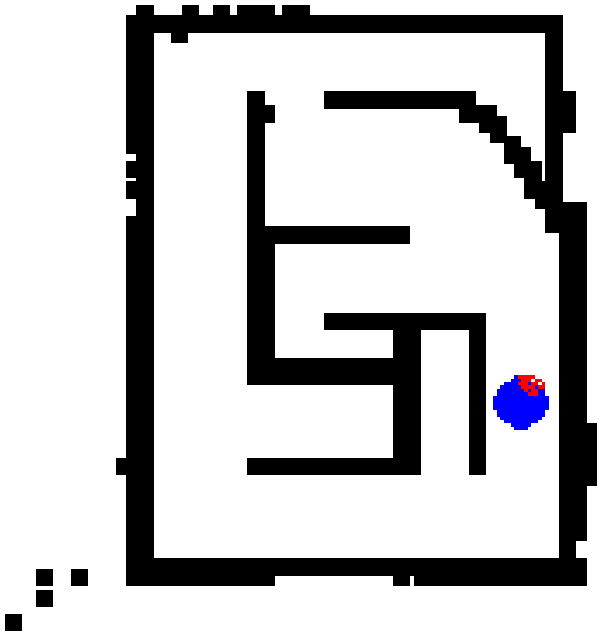
BotLab Part 2



Botlab Simulator

The botlab simulator provides a means to test your code without a robot. The simulator models motor commands to robot motion, robot motion to odometry readings, LIDAR, robot collisions with a grid cell based map, and simple sensor noise. You should think of the simulator as the real world, it shows you what the robot is actually doing whereas the botgui shows you what the robot thinks about the world.

Running the Simulator

The simulator should be run from botlab/src/sim as follows:

```
>$ cd <path_to_botlab>/src/sim
>$ python3 sim.py <path_to_.map_file>
```

To use with the rest of botlab software run the following:

```
>$ cd <path_to_botlab>/src/sim
>$ ./timesync
>$ ./botgui
>$ ./motion_controller
>$ ./slam
```

Note, it is best to run each of these in their own separate terminal window so you can clearly see error messages and output messages.

LCM Interface

The simulator is designed to be controlled through lcm messages. To move the robot around and get feedback from the simulator is all controlled through lcm channels.

The lcm interface is initialized with "udpm://239.255.76.67:7667?ttl=2". The following channels are published:

- ODOMETRY - odometry_t.lcm, the simulated robot odometry
- LIDAR - lidar_t.lcm, the simulated lidar scans

The following channels are subscribed to:

- MBOT_MOTOR_COMMAND - mbot_motor_command_t.lcm, these messages are used to control the motion of the simulated MBot

Please see the technical details section to learn more about the behavior of the simulator and how it treats these channels.

Configuration

The simulator has several command line options that adjust the behavior of the simulator. Run the following command to see a description of the available options:

```
>$ python3 sim.py --help
```

The following arguments are available:

- map_file - a required positional argument that specifies the map the simulator should use
- render_lidar - renders the lidar in the simulator if true
- use_noise - Add noise to lidar and odometry if true
    - lidar_dist_measure_sigma - Standard deviation of a 0 mean gaussian distribution used to add random noise to the lidar distance measurements
    - lidar_theta_step_sigma - Standard deviation of a 0 mean gaussian distribution used to add random noise to the angles between each lidar scan
    - lidar_num_ranges_noise - Half the size of a uniform distribution centered at 0 over the integers used to randomize the number of lidar measurements
    - odom_trans_sigma - Standard deviation of a 0 mean gaussian distribution used to add random noise to the odometry translation
    - odom_rot_sigma - standard deviation of a 0 mean gaussian distribution used to add random noise to the odometry rotation

- width - The width in pixels of the simulator (height determined based on .map file)
- mbot_max_trans_speed - The mbots maximum translation speed
- mbot_max_angular_speed - the mbots maximum angular speed

Implementing Planning and Exploration

Using the SLAM algorithm implemented in part 1, you can now construct a map of an environment using the MBot simulator. Now you will implement additional capabilities for the MBot: path planning using A* and autonomous exploration of an environment.

2.1 Obstacle Distance:

The configuration space is located in `planning/obstacle_distance_grid.h/.cpp`. Run `obstacle_distance_grid_test` and check if your code passes the three tests. The botgui already has a call for a mapper object from which it can obtain and draw an obstacle distance grid. You can view the generated obstacle distance grid by ticking "Show Obstacle Distances" in botgui.

Deliverables:

- For the report: Comment on whether your code passed or failed the three tests in obstacle_distance_grid_test. Provide remarks on the code's performance and whether there is anything major slowing it down.
- Play the log for obstacle_slam_10mx10m_5cm.log, tick the "Show Obstacle Distances" option in botgui, and provide a snapshot of the final obstacle distance grid obtained at the end of the log's playback. Provide the image within your report.

2.2 A* Path Planning:

Write an A* path planner that will allow you to find plans from one pose in the environment to another. You will integrate this path planner with the `motion_controller` from Phase 2 to allow your MBot to navigate autonomously through the environment.

For this phase, you will implement an A* planner and a simple configuration space for the MBot. The skeleton for the A* planner is located in `planning/astar.h/.cpp` Your implementation will be called by the code in `planning/motion_planner.h/.cpp`

Once your planner is implemented, test it by constructing a map using your SLAM algorithm and then using botgui to generate a plan. Right-click somewhere in free space on your map. Your planner will then be run inside botgui and a robot_path_t will be sent to the motion_controller for execution.

`astar_test` & `astar_test_files` can be used to test the performance of your A* planner.

Deliverables:

- For the report: Using `astar_test_files`, please report statistics on your path planning execution times for each of the example problems in the `data/astar` folder -- you simply need to run `astar_test_files` after implementing your algorithm. If your algorithm is optimal and fast, great. If not please discuss possible reasons and strategies for improvement.

- Setup the simulator to run with the map for Run2_maze_hires/Maze_hires_6_6_2.map from the botlab-data repo, and run ./slam in localization-only mode. Select an arbitrary goal point of your choosing on the map by Right-Clicking a free cell on your botgui. Your planner should be able to use Astar to plan to the point and provide robot_path_t messages to your motion controller. Provide a log showing the path planning and the robot driving to the objective.

2.3 Map Exploration:

Up to now, your MBot has always been driven by hand or driven to goals selected by you. For this task, you'll write an exploration algorithm that will have the MBot autonomously select its motion targets and plan and follow a series of paths to fully explore an environment.

We have provided an algorithm for finding the frontiers -- the borders between free space and unexplored space -- in `planning/frontiers.h/.cpp`. Plan and execute a path to drive to the frontier. Continue driving until the map is fully explored, i.e. no frontiers exist. Once you have finished exploring the map, return to the starting position. Your robot needs to be within 0.05m of the starting position. The state machine controlling the exploration process is located in `planning/exploration.h/.cpp`.

Deliverables:

- Setup the simulator to run with the map for Run2_maze_hires/Maze_hires_6_6_2.map from the botlab-data repo. Start a logger session and run full ./slam followed by ./exploration. Your robot should be able to plan paths to the frontiers present, exploring the map in the process, until no frontiers exist, before finally returning to the home position. Provide a log of the simulated mbot executing an exploration process and returning to home position.

- Comment on your exploration performance
  - What are the factors that are preventing your exploration from being ideal?
  - Provide your logic behind tackling frontiers (how do you go about chosing your next frontier to go to)
  - Comments on the performance of Astar and path planning with automated exploration commands.

Submission Instructions

Please zip and submit your project as given below. Points will be deducted for improper submission.

```
name = your umich shortname
grid_size = <grid_height>mx<grid_width>m
cell_size = <size>cm

Example log file name for a 8mx8m grid, 4cm cell size:  drive_square_8mx8m_4cm.log
Note: all deliverables for 2_1 are in the report.

<name>-botlab-w20.zip
    -   <name>-botlab-f19/
          -   code/
                  -   <name>-botlab-f19/
          -   logs/
                  -   1_1/
                          -   mapping_convex_<grid_size>_<cell_size>.log
                          -   mapping_drive_square_<grid_size>_<cell_size>.log
                          -   mapping_maze_lowres_<grid_size>_<cell_size>.log
                  -   1_2_1/
                          -   action_drive_square_<grid_size>_<cell_size>.log
                          -   action_obstacle_slam_<grid_size>_<cell_size>.log
                          -   action_straight_line_<grid_size>_<cell_size>.log
                  -   1_2_2/
                          -   sensor_drive_square_<grid_size>_<cell_size>.log
                          -   sensor_obstacle_slam_<grid_size>_<cell_size>.log
                          -   sensor_straight_line_<grid_size>_<cell_size>.log
                  -   1_3/
                          -   full_obstacle_slam_<grid_size>_<cell_size>.log
                          -   full_maze_lowres_<grid_size>_<cell_size>.log
                          -   full_maze_hires_<grid_size>_<cell_size>.log
                          -   full_[Bonus] maze_hard_<grid_size>_<cell_size>.log
                  -   2_2/
                          -   astar_maze_hires_<grid_size>_<cell_size>.log
                  -   2_3/
                          -   explore_maze_hires_<grid_size>_<cell_size>.log
          -   report/
                  -   <name>-botlab-report.pdf

You can access the report template HERE.
```